

Devops

1 Définition de DevOps :

Le terme DevOps **est une contraction** de “**Development**” (Développement) et “**Operations**” (Opérations).

C'est une culture, une méthodologie et un ensemble de pratiques qui visent à rapprocher les équipes de développement logiciel (Dev) et les équipes d'exploitation / opérations (Ops) pour :

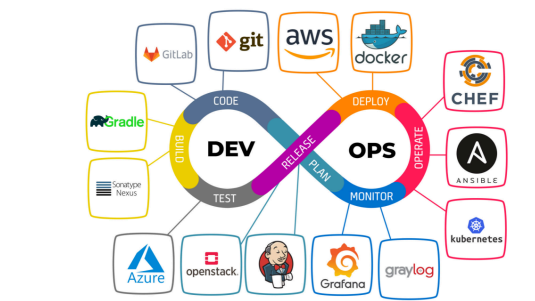
Les principes DevOps (CALMS) :

- **Culture** : Comprendre et améliorer les valeurs et les attitudes de l'environnement d'entreprise au service du développement.
 - **Automation** : Tout ce qui peut être automatisé doit l'être.
 - **Lean** : Économie de coûts et élimination des ressources inutiles.
 - **Mesure** : Localiser rapidement les erreurs logicielles et analyser le comportement des utilisateurs par la mise en place de remontées d'informations.
 - **Sharing** (partage) : Partage et communication entre les équipes.
- ♦ **DevOps** est un ensemble de pratiques, d'outils et une **philosophie** culturelle qui visent à **optimiser la collaboration et l'automatisation** entre les équipes de développement logiciel (**Dev**) et d'exploitation informatique (**Ops**). Son objectif principal est de **livrer des logiciels de manière plus rapide, fiable et sécurisée**.

2 Les avantages de DevOps

En adoptant DevOps, une entreprise gagne :

- ✓ **Déploiements plus rapides** – passer de quelques mois à quelques jours/heures
- ✓ **Moins d'erreurs** grâce à l'automatisation des tests et des déploiements
- ✓ **Collaboration renforcée** entre les équipes
- ✓ **Meilleure satisfaction client** (produits plus stables et plus fréquents)
- ✓ **Réduction des coûts** liés aux bugs et aux incidents
- ✓ **Plus d'agilité** pour répondre aux besoins du marché



2 Les environnements DevOps



Les principaux environnements DevOps sont les

suivants :

- **Environnement de développement (DEV)** : C'est l'environnement où les développeurs écrivent et testent le code. Il est généralement isolé des autres environnements pour éviter les conflits et les problèmes de sécurité.
- **Environnement d'intégration (INT)** : C'est l'environnement où le code de **plusieurs** développeurs est **intégré** et **testé ensemble**. Il permet de détecter **les problèmes d'intégration** précocement et de les corriger avant qu'ils ne se propagent aux autres environnements.
- **Environnement de test (UAT)** : C'est l'environnement où les utilisateurs **finaux** testent le code **avant qu'il ne soit déployé en production**. Il permet de s'assurer que le code répond aux besoins des utilisateurs et qu'il est exempt de bogues.
- **Environnement de qualification (QUALIF)** : C'est l'environnement où le code est testé dans un **environnement de production simulé**. Il permet de s'assurer que le code fonctionne correctement dans un environnement réel avant qu'il ne soit déployé en production.
- **Environnement de pré-production (PREPROD)** : C'est l'environnement où le code est déployé avant d'être mis à disposition des utilisateurs finaux. Il permet de tester le code dans un environnement **réel** et de résoudre les derniers problèmes avant le déploiement en production.
- **Environnement de production (PROD)** : C'est l'environnement où le code est déployé et mis à disposition des utilisateurs finaux. Il doit être stable et fiable afin de garantir une expérience utilisateur optimale.

💡 Exemple global :

Prenons l'exemple d'une **application bancaire** :

1. Les développeurs codent la fonctionnalité "virement instantané" → **DEV**
2. On intègre le code de toute l'équipe → **INT**
3. Les testeurs valident les cas d'usage → **UAT**
4. On simule des milliers d'utilisateurs → **QUALIF**
5. On vérifie la stabilité sur un serveur miroir → **PREPROD**
6. La fonctionnalité est disponible pour les clients → **PROD**

3 Définition de CI/CD

♦ **CI (Continuous Integration) = Intégration Continue**

- Chaque fois qu'un développeur **pousse du code** (commit/push) dans le dépôt Git,
- Le système CI (Jenkins, GitLab CI, GitHub Actions...) **compile, teste et vérifie** automatiquement le code.

⇒ **L'intégration continue (CI)** est une pratique qui consiste à automatiser **l'intégration** des modifications de code provenant de différents développeurs dans un référentiel central. Cela permet de détecter et de corriger les problèmes plus tôt dans le processus de développement.

- Objectif : **détecter rapidement** les erreurs et assurer que tout le monde travaille sur un code stable.

♦ **CD (Continuous Delivery / Continuous Deployment) = Livraison/Déploiement Continu**

- **Continuous Delivery** = Le code validé est **préparé automatiquement** pour être déployé (package, Docker image...), et **un clic suffit pour le mettre en production**.
- **Continuous Deployment** = Le code validé est **déployé automatiquement** en production **sans intervention manuelle**.
- Objectif : **livrer rapidement** des mises à jour stables et testées aux utilisateurs.

4 Les étapes d'un pipeline CI/CD typique

1. Commit du code sur Git (GitHub, GitLab...)
2. Build automatique (compiler, générer artefacts)
3. Tests automatiques (unitaires, sécurité, performance...)
4. Création du package (Docker, .jar, etc.)
5. Déploiement automatique sur un environnement de test (staging)
6. Déploiement automatique en production (si tout est OK)

🌀 Le cycle DevOps (Pipeline CI/CD)

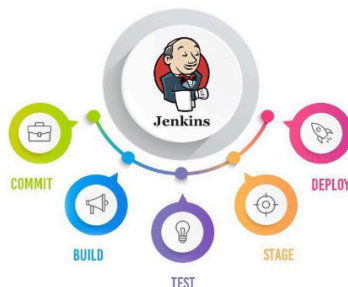
Le cycle DevOps est souvent représenté sous forme d'un ∞ (**infini**) parce qu'il est **continu** : on développe, on teste, on déploie, on surveille... puis on recommence :

Étape	Action	Objectif
Planifier	Définir les fonctionnalités	Alignement
Développer	Écrire le code	Propreté & version
Intégration Continue (CI)	Build + tests automatiques	Détection rapide des bugs

Étape	Action	Objectif
Tester	QA automatique	Qualité
Packager	Docker / artefact	Standardiser
Déploiement Continu (CD)	Automatiser le déploiement	Rapidité & fiabilité
Surveiller	Logs & métriques	Réactivité
Améliorer	Boucle de feedback	Progression continue

❖ Outils d'intégration continue

Jenkins : Un serveur open source écrit en Java qui automatise l'intégration continue.



o Architecture distribuée de Jenkins

L'une des principales caractéristiques qui font de **Jenkins** un outil d'intégration continue robuste est son architecture **maître-esclave** (master-slave), permettant la distribution des tâches de construction et de déploiement sur plusieurs machines.

Le serveur Jenkins (master) délègue les tâches aux **agents** (slaves), ce qui permet de distribuer les charges de travail (builds).

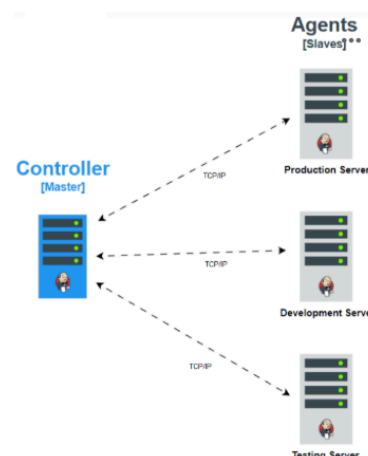
- ♦ Le **Master (Contrôleur)** décide **quoi faire et quand**.
- ♦ Les **Agents (Nœuds)** exécutent **les actions demandées**.

Élément	Rôle principal	Fonctions détaillées	Exemple concret
 Master (Contrôleur Jenkins)	Chef d'orchestre du système Jenkins	- Gère la configuration du pipeline CI/CD- Déclenche les builds automatiquement- Distribue les tâches aux agents (nœuds)- Surveille les résultats des builds et envoie les rapports	Le contrôleur détecte qu'un nouveau code a été poussé sur GitHub et lance le pipeline d'intégration.

Élément	Rôle principal	Fonctions détaillées	Exemple concret
 Agent (Nœud Jenkins)	Machine d'exécution des tâches	- Exécute les tâches envoyées par le contrôleur (tests, builds, déploiements)- Peut être dédié à un type de tâche (ex : tests unitaires, création d'images Docker)- Retourne les résultats au contrôleur	Un agent exécute les tests unitaires pendant qu'un autre construit une image Docker.

Le workflow CI, en un environnement distribué de build, se constitue d'une suite d'étapes à suivre, à savoir :

1. Le développeur commit les modifications
2. Le **contrôleur** Jenkins vérifie le référentiel
3. Le contrôleur Jenkins distribue la charge de travail (les builds) aux agents Jenkins
4. Les agents exécutent les tâches qui leurs sont demandées.
5. Le contrôleur Jenkins surveille l'exécution de builds



♦ Comment Jenkins s'intègre avec Git dans DevOps

1. **Jenkins est connecté à ton projet Git (GitHub, GitLab, Bitbucket, etc.)**
 - On configure dans Jenkins l'URL de ton dépôt Git.
 - On peut aussi mettre un **webhook** dans GitHub pour prévenir Jenkins quand il y a un nouveau commit ou une nouvelle branche.
2. **À chaque modification du code (push, merge, nouvelle branche)**
 - Git envoie un signal à Jenkins (via **webhook**).
 - Jenkins va automatiquement **tirer (pull) le code** de ton dépôt.
3. **Jenkins lance le pipeline (script d'automatisation)**
 - **Build** : compile ton code (ex: mvn package pour Java, npm install pour Node.js...).
 - **Tests** : exécute les tests unitaires, d'intégration ou fonctionnels.
 - **Analyse** : vérifie la qualité du code avec **SonarQube** par exemple.

❖ Jenkinsfile :

🧩 Définition

Un **Jenkinsfile** est un fichier texte écrit en **Groovy** qui décrit le **pipeline Jenkins**, c'est-à-dire les étapes (stages) de l'intégration et du déploiement continu d'un projet.

Il existe deux types de syntaxe :

- **Déclarative pipeline** (basée sur des mots-clés)
- **Scripted pipeline** (basée sur Groovy pur)
-  **Structure générale d'un Jenkinsfile**

```
pipeline {
  agent any
  tools { ... }
  options { ... }
  environment { ... }
  stages { ... }
  post { ... }
}
```

Principales directives:

Directive	Rôle	Exemple
pipeline	Bloc principal contenant tout le code du pipeline.	pipeline { ... }
agent	Définit où le pipeline s'exécute (sur quelle machine/agent Jenkins).	agent any ou agent { node { label 'build' } }
tools	Spécifie les outils nécessaires (ex : Maven, JDK, etc.).	tools { maven 'M2_HOME' }
options	Définit des paramètres globaux (ex : timeout, rétention de logs...).	options { timeout(time: 1, unit: 'SECONDS') }
environment	Définit des variables d'environnement accessibles dans tout le pipeline.	environment { APP_ENV = "DEV" }
stages	Regroupe toutes les étapes du pipeline. Chaque <i>stage</i> contient des <i>steps</i> .	stages { stage('Build') { steps { ... } } }
post	Contient des actions exécutées à la fin du pipeline (succès, échec, toujours...).	post { success { ... } failure { ... } }

“exp”

```
pipeline {
  agent any

  tools {
    maven 'M2_HOME'
  }

  options {
    timeout(time: 1, unit: 'SECONDS')
  }

  environment {
    APP_ENV = "DEV"
  }

  stages {
    stage('Code Checkout') {
      steps {
```

```
        stage('Code Build') {
          steps {
            sh 'mvn install -Dmaven.test.skip=true'
          }
        }
      }
    }
  }

  post {
    always {
      echo "————always————"
    }

    success {
      echo "————pipeline executed successfully————"
    }

    failure {
      echo "————pipeline execution failed————"
    }
  }
}
```

1. Structure générale d'un Jenkinsfile

```
pipeline {  
    agent any          // sur quel agent exécuter le pipeline  
  
    tools {            // les outils nécessaires (optionnel)  
        maven 'Maven-3.8.6'  
        nodejs 'Node-18'  
    }  
  
    environment {      // les variables d'environnement globales  
        APP_NAME = 'MyApp'  
        DOCKERHUB_USER = 'syrine123'  
    }  
  
    stages {           // les étapes du pipeline  
        stage('Checkout') { ... }  
        stage('Build') { ... }  
        stage('Test') { ... }  
        stage('Deploy') { ... }  
    }  
  
    post {             // actions à la fin du pipeline (optionnel)  
        always { echo 'Pipeline terminé.' }  
    }  
}
```

5. But de chaque stage

Stage	But principal	Actions typiques
 Checkout	Récupérer le code source	git clone depuis GitHub ou GitLab
 Build	Compiler / construire le projet	npm run build, mvn package, gradle build
 Test	Exécuter les tests automatisés	Tests unitaires et d'intégration

Stage	But principal	Actions typiques
 Docker Build & Push	Créer et envoyer l'image Docker vers un registre	docker build, docker push
 Deploy	Déployer l'application sur un serveur ou cloud	ssh, kubectl apply, ou ansible-playbook
 Notify (optionnel)	Informer les équipes	Envoi d'email ou message Slack après succès/échec

Docker

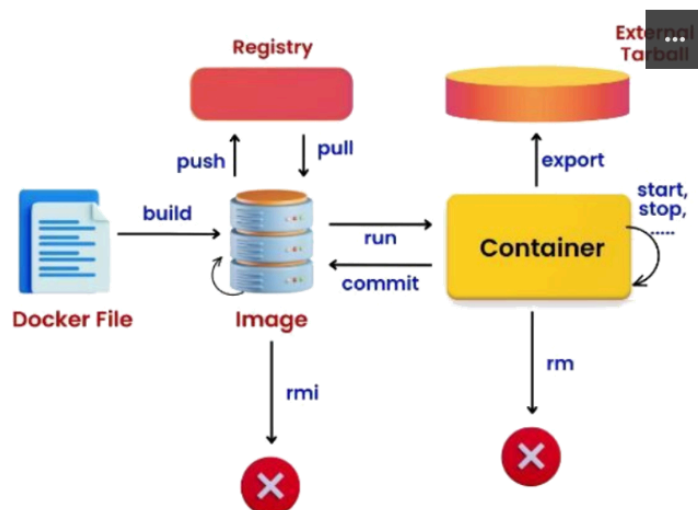
1 Qu'est-ce que Docker ?

Docker est une plateforme de virtualisation légère qui permet de créer, déployer et exécuter des applications dans des conteneurs.

Caractéristiques principales :

- Isolation : chaque conteneur est indépendant, avec son propre système de fichiers et ses dépendances.
- Léger : contrairement aux machines virtuelles, les conteneurs partagent le noyau du système hôte.
- Portable : un conteneur fonctionne de la même manière sur n'importe quelle machine.
- Reproductible : l'environnement est identique pour tous les développeurs et serveurs.

Docker : Workflow et Commandes



Étape	Description	Commande principale	Objectif
1. Dockerfile → Image	Création d'une image personnalisée à partir d'un fichier Dockerfile.	<code>docker build -t nom_image .</code>	Construire une image Docker.
2. Image → Registry	Envoi de l'image vers un registre Docker (Docker Hub, GitLab, etc.).	<code>docker push nom_registry/nom_image</code>	Partager ou sauvegarder l'image dans un registre.
3. Registry → Image	Téléchargement d'une image depuis un registre Docker.	<code>docker pull nom_registry/nom_image</code>	Récupérer une image existante pour l'utiliser localement.

Étape	Description	Commande principale	Objectif
4. Image → Container	Création et exécution d'un conteneur à partir d'une image.	<code>docker run nom_image</code>	Démarrer un conteneur basé sur l'image.
4 bis. Container → Nouvelle Image	Sauvegarde d'un conteneur modifié sous forme d'une nouvelle image.	<code>docker commit id_conteneur nom_nouvelle_image</code>	Conserver les modifications d'un conteneur.
5. Container → Fichier .tar	Exportation du système de fichiers du conteneur vers une archive externe.	<code>docker export -o nom_fichier.tar id_conteneur</code>	Sauvegarder le contenu d'un conteneur sans ses métadonnées.
6. Gérer le Conteneur	Gestion du cycle de vie des conteneurs.	<code>docker start / docker stop / docker rm</code>	Démarrer, arrêter ou supprimer un conteneur.
7. Supprimer une Image	Suppression d'une image locale non utilisée.	<code>docker rmi nom_image</code>	Libérer de l'espace disque et nettoyer les images inutiles.

Exemple rapide

```
# Télécharger l'image Ubuntu
docker pull ubuntu

# Lancer un conteneur interactif
docker run -it ubuntu

# Dans le conteneur : vérifier la version
lsb_release -a

# Quitter le conteneur
exit

# Lister les conteneurs en cours et tous les conteneurs
docker ps
docker ps -a

# Supprimer un conteneur arrêté
docker rm <container_id>
```

Résumé :

Docker permet de **packager et exécuter des applications dans des conteneurs** de manière

portable et reproductible. Les commandes de base te permettent de **télécharger des images, lancer des conteneurs, gérer les volumes et les réseaux.**

Container Management	General Management	Image Management
<code>docker ps</code> List the running containers.	<code>docker login</code> Log in to a Docker registry.	<code>docker build [dockerfile-path]</code> Create an image from a Dockerfile.
<code>docker ps -a</code> List all the containers, both running and stopped.	<code>docker logout</code> Log out of a Docker registry.	<code>docker build .</code> Build an image using the files from the current path.
<code>docker create [image]</code> Create a container without starting it.	<code>docker inspect [object]</code> Show low-level information about an object.	<code>docker build -t [name]:[tag] [location]</code> Create an image from a Dockerfile and tag it.
<code>docker create -it [image]</code> Create an interactive container with pseudo-TTY.	<code>docker version</code> Show the version of the local Docker installation.	<code>docker build -f [file]</code> Specify a file to build from.
<code>docker rename [container] [new-name]</code> Rename a container.	<code>docker info</code> Display information about the system.	<code>docker pull [image]</code> Pull an image from a registry.
<code>docker rm [container]</code> Remove a stopped container.	<code>docker system prune</code> Remove unused images, containers, and networks.	<code>docker push [image]</code> Push an image to a registry.
<code>docker rm -f [container]</code> Force remove a container, even if it is running.		<code>docker import [url/file]</code> Create an image from a tarball.
<code>docker logs [container]</code> View logs for a running container.		<code>docker commit [container] [new-image]</code> Create an image from a container.
<code>docker logs -f --until=[interval] [container]</code> Retrieve logs before a specific point in time.		<code>docker tag [image] [image]:[tag]</code> Tag an image.
<code>docker events [container]</code> View real time events for a container.		<code>docker images</code> Show all locally stored top level images.
<code>docker update [container]</code> Update the configuration of a container.		<code>docker history [image]</code> Show history for an image.
<code>docker port [container]</code> Show port mapping for a container.		<code>docker rmi [image]</code> Remove an image.
<code>docker top [container]</code> Show running processes in a container.		<code>docker load --image [tar-file]</code> Load an image from a tar archive file.
<code>docker stats [container]</code> Show live resource usage statistics for a container.		<code>docker save [image] > [tar-file]</code> Save an image to a tar archive file.
<code>docker diff [container]</code> Show changes to files or directories on the filesystem.		<code>docker search [query]</code> Search Docker Hub for images.
<code>docker cp [file-path] CONTAINER:[path]</code> Copy a local file to a directory in a container.		<code>docker image prune</code> Remove unused images.
Networking	Running a Container	Plugin Management
<code>docker network ls</code> View available networks.	<code>docker run [image] [command]</code> Run a command in a container based on an image.	<code>docker plugin enable [plugin]</code> Enable a Docker plugin.
<code>docker network rm [network]</code> Remove a network.	<code>docker run --name [container-name] [image]</code> Create, start, and name a container.	<code>docker plugin disable [plugin]</code> Disable a Docker plugin.
<code>docker network inspect [network]</code> Show information about a network.	<code>docker run -p [host]:[container-port] [image]</code> Map a host port to a container port.	<code>docker plugin create [plugin] [path-to-data]</code> Create a plugin from config.json and rootfs.
<code>docker network connect [network] [container]</code> Connect a container to a network.	<code>docker run --rm [image]</code> Run a container and remove it after it stops.	<code>docker plugin inspect [plugin]</code> View details about a plugin.
<code>docker network disconnect [network] [container]</code> Disconnect a container from a network.	<code>docker run -d [image]</code> Run a detached (background) container.	<code>docker plugin rm [plugin]</code> Remove a plugin.
	<code>docker run -it [image]</code> Run an interactive process, e.g., a shell, in a container.	
	<code>docker start [container]</code> Start a container.	
	<code>docker stop [container]</code> Stop a container.	
	<code>docker restart [container]</code> Stop a container and start it again.	
	<code>docker pause [container]</code> Pause processes in a running container.	
	<code>docker unpause [container]</code> Unpause processes in a running container.	
	<code>docker wait [container]</code> Block input until the container stops.	
	<code>docker kill [container]</code> Send a SIGKILL signal to stop a container.	
	<code>docker attach [container]</code> Attach local standard input, output and error.	
	<code>docker exec -it [container] [shell]</code> Run a shell inside a running container.	

⚙️ 2. Pourquoi installer Docker et Docker Compose ?

Parce que Jenkins **doit créer une image Docker** de ton application après l'étape "Build".

👉 Docker sert à :

- **Emballer** ton application avec toutes ses dépendances (Java, Node.js, etc.) dans une **image**.
- **Rendre ton appli portable** (elle peut tourner n'importe où : sur ton PC, ton serveur, ton cloud...).

Exemple d'étape Jenkins :

```
stage('Docker Build') {
    steps {
        sh 'docker build -t monapp:latest .'
```

```
}  
}
```

🗨 Ici Jenkins dit à Docker :

“Crée-moi une image nommée **monapp** à partir du Dockerfile du projet.”

👉 **Docker Compose sert à :**

- **Lancer plusieurs conteneurs ensemble**, par exemple :
 - ton application (monapp)
 - une base de données (mongo, mysql, etc.)
 - un serveur backend et frontend ensemble

Exemple :

```
stage('Deploy with Docker Compose') {  
    steps {  
        sh 'docker-compose up -d'  
    }  
}
```

🗨 Ici Jenkins :

“Démarré tous les services définis dans le fichier docker-compose.yml.”

🧠 3. En résumé

Outil	Rôle dans Jenkins
Docker	Créer et exécuter des conteneurs (images de ton app)
Dockerfile	Décrit comment construire ton image
Docker Compose	Lance plusieurs conteneurs ensemble (app + DB + outils)
Jenkins	Orchestration : appelle Docker pour builder, tester et déployer

un résumé structuré :

1. **But du Dockerfile** : Créer une image contenant ton application et toutes ses dépendances.

2. Structure d'un Dockerfile :

- o **FROM** : Spécifie l'image de base (par exemple, openjdk:11 pour une application Java).
- o **WORKDIR** : Définit le répertoire de travail dans le conteneur.
- o **COPY** : Copie les fichiers de ton application dans l'image.
- o **RUN** : Exécute des commandes, par exemple pour installer des dépendances.
- o **CMD** ou **ENTRYPOINT** : Définit la commande qui s'exécute au démarrage du conteneur.

3. Exemple simplifié :

4. FROM openjdk:11
5. WORKDIR /app
6. COPY . /app
7. RUN ./mvnw clean package
8. CMD ["java", "-jar", "target/myapp.jar"]

Ainsi, le Dockerfile permet de bâtir une image reproductible et portable.

Il vaut mieux utiliser une image officielle JDK (comme openjdk:17-jdk-slim) car :

-  Elle contient déjà Java (JDK ou JRE) configuré correctement.
-  Elle est optimisée pour exécuter des applications Java (plus légère et plus rapide que Ubuntu).
-  Elle est maintenue et sécurisée par la communauté OpenJDK (mises à jour régulières).
-  Elle réduit les erreurs de compatibilité et les étapes d'installation.

Exemple complet de pipeline simplifié :

pipeline {

agent any

stages {

stage('Checkout') {

steps { git branch: 'main', url: 'https://github.com/...' }

}

stage('Build') {

steps { sh 'mvn clean package' }

}

stage('Docker Build') {

```

    steps { sh 'docker build -t monapp:latest.' }
  }
  stage('Deploy') {
    steps { sh 'docker-compose up -d' }
  }
}
}

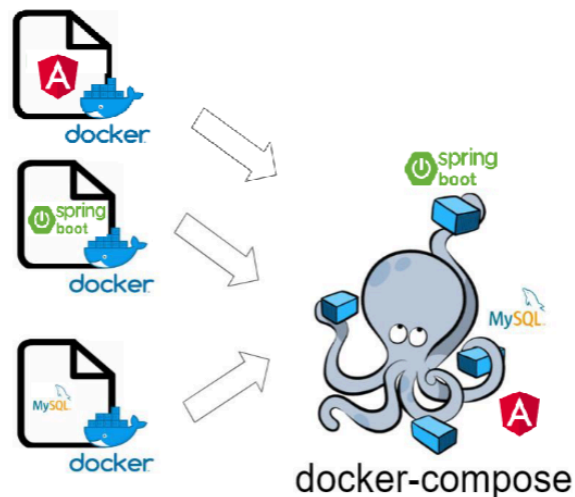
```

✚ Donc pour résumer :

Tu installes **Docker et Docker Compose** sur le serveur Jenkins car :

- Jenkins doit **construire** ton image (Docker)
- et souvent **déployer** ton appli avec d'autres services (Docker Compose)

👉 **Docker Compose** est utilisé dans Jenkins et dans les pipelines CI/CD (Continuous Integration / Continuous Delivery) pour **automatiser le déploiement de plusieurs services Docker qui fonctionnent ensemble.**



✚ 1. Rôle de Docker Compose

Docker Compose te permet de :

- Définir **plusieurs** conteneurs dans **un seul fichier (docker-compose.yml)**
- Démarrer tous ces conteneurs avec une seule commande :

- docker-compose up -d
- Gérer facilement les dépendances entre services (exemple : Jenkins a besoin d'un conteneur MySQL ou d'un serveur web pour fonctionner).

⚙️ 2. Utilisation dans Jenkins

Dans Jenkins :

- On utilise **Docker Compose** pour **lancer l'environnement d'intégration** avant d'exécuter les tests.
- Exemple : tu as une application web + une base de données → Jenkins utilise docker-compose.yml pour **les démarrer ensemble** avant de lancer les tests.
- les 3 fonctions principales de docker-compose sont :
 - Comment lancer un docker-compose? (se mettre dans le dossier contenant le fichier docker-compose.yml) :
docker compose up -d
 - Comment vérifier les logs des conteneurs qui ont été lancé?
docker compose logs
 - Comment arrêter un docker compose ?
docker compose down

♦ Exemple dans une pipeline Jenkins :

```
stage('Deploy with Docker Compose') {
  steps {
    sh 'docker-compose up -d'
  }
}
```

Exemple de **docker-compose.yml** :

```
version: '3'

services:
  app:
    build: .
    ports:
      - "3000:3000"
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
```

Puis dans Jenkinsfile :

```
pipeline {
  agent any

  stages {
    stage('Build & Test') {
      steps {
        sh 'docker-compose up -d'
        sh 'docker-compose run app npm test'
        sh 'docker-compose down'
      }
    }
  }
}
```

MYSQL_DATABASE: mydb

👉 Ici, Jenkins :

Lance les conteneurs avec docker-compose up -d

Fait tourner les tests

🌿 Exemple simplifié d'utilisation combinée :

```
pipeline {
  agent any
  stages {
    stage('Build App Image') {
      steps {
        sh 'docker build -t myapp .'
      }
    }
    stage('Run Integration Tests') {
      steps {
        sh 'docker-compose up -d'
        sh 'docker exec myapp-container mvn test'
        sh 'docker-compose down'
      }
    }
    stage('Deploy') {
      steps {
        sh 'docker-compose -f docker-compose.prod.yml up -d'
      }
    }
  }
}
```

1. Stoppe tout avec docker-compose down

Docker Compose CLI Cheat sheet

docker-compose build docker-compose build [options] [SERVICE...] Use: Builds docker images for the listed services in compose file. Flag: --no-cache avoids image cache while building the docker image. docker-compose build --pull nginx-service It will pull the latest version of nginx image and build the latest docker image for nginx-service.	docker-compose down docker-compose down [options] Use: Stops all running containers, images, volumes, networks initiated by docker-compose up command. Flag: --remove-orphans removes containers for services not defined in compose file. docker-compose down -v It will remove both named volumes (mentioned in the compose file) and anonymous volumes attached to containers.	docker-compose exec docker-compose exec [options] SERVICE [COMMAND] [ARGS...] Use: Runs new commands in a running service container with default pseudo-TTY allocation. Flag: --privileged gives extended privileges to processes. docker-compose exec ubuntu-service bash It will run a new bash session in the default directory of the running ubuntu-service container.
docker-compose run docker-compose run [options] SERVICE [COMMAND] [ARGS...] Use: Runs a one-time command in a new container with same configurations of the mentioned service. Flag: --entrypoint CMD overrides the entrypoint of the image. docker-compose run ubuntu-service bash It will run a bash session in a new container, with the same configurations of running ubuntu-service.	docker-compose kill docker-compose kill [options] [SERVICE...] Use: Forces running containers to stop with SIGKILL. Flag: -s [SIGNAL_NAME] passes different types of signals to running containers. docker-compose kill -s SIGTERM It will send a termination signal to the all the compose initiated running containers.	docker-compose logs docker-compose logs [options] [SERVICE...] Use: Displays log output from services. Flag: --tail="10" shows the last 10 lines of logs from each containers. docker-compose logs -t >> compose-logs.txt It will get logs of all compose initiated containers with their respective timestamps and save it to the "compose-logs.txt" file.
docker-compose ps docker-compose ps [options] [SERVICE...] Use: Lists out existing containers along with their properties. Flag: -q/--quite shows only the list of container IDs. docker-compose ps --all It will display all running as well as stopped containers initiated by docker compose with containers' properties such as container IDs, default execution commands, state of container, etc.	docker-compose up docker-compose up [options] [SERVICE...] Use: Builds, (re)creates, starts, and attaches to containers for a service. Flag: -d/--detach runs containers in detach mode. docker-compose up -d It will start containers in background and keeps them running.	docker-compose rm docker-compose rm [options] [SERVICE...] Use: Removes all stopped containers for services. Flag: -v removes any anonymous volume attached to the containers. docker-compose rm --force It will remove all the stopped service containers without asking for user's approval.
	docker-compose help docker-compose help COMMAND Use: Shows user instructions and details for a particular command. docker-compose help up It will show usage instruction and option details for up command.	

🧩 1. Les fichiers nécessaires

Pour que Jenkins puisse construire et exécuter ton application dans Docker, il te faut **deux fichiers principaux** :

- ♦ a) **Dockerfile**

C'est le fichier qui définit l'**image Docker de ton application** (par exemple ton projet Java, Node, React...).

- ♦ b) **docker-compose.yml**

C'est le fichier qui **orchestré** plusieurs conteneurs ensemble :
ton application + base de données + serveur web, etc.

📁 2. Où placer ces fichiers ?

Tu dois les placer **à la racine de ton projet**, c'est-à-dire dans le même dossier que ton code source et ton fichier pom.xml (si tu utilises Maven).

Exemple d'arborescence :

```
mon-projet/
|
|— src/
|  |— main/
|  |— test/
|
|— pom.xml
|— Dockerfile
|— docker-compose.yml
|— Jenkinsfile
```

💡 Pourquoi à la racine ?

Parce que Jenkins va **cloner ton dépôt Git** et exécuter les commandes Docker depuis ce dossier. Ainsi, il trouvera directement les fichiers nécessaires.

🧱 3. Contenu du Dockerfile (exemple Maven/Java)

Si ton projet utilise **Maven + Spring Boot**, ton Dockerfile peut ressembler à ceci :

```
# Étape 1 : Build avec Maven
FROM maven:3.9.4-eclipse-temurin-17 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Étape 2 : Exécuter l'application
FROM eclipse-temurin:17-jdk
WORKDIR /app
```

```
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

 Ce que ça fait :

1. Télécharge une image Maven pour compiler ton projet.
2. Crée le .jar dans /target.
3. Copie ce .jar dans une image finale (plus légère) et le lance avec Java.

 4. Contenu du docker-compose.yml (exemple)

Si ton application a besoin d'une **base de données MySQL**, par exemple :

```
version: '3.8'
services:
  app:
    build: .
    container_name: myapp-container
    ports:
      - "8080:8080"
    depends_on:
      - db
  db:
    image: mysql:8
    container_name: mysql-container
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mydb
    ports:
      - "3306:3306"
```

 Ce que ça fait :

- Lance **ton application** à partir du Dockerfile.
- Lance aussi une **base de données MySQL**.
- Connecte les deux conteneurs ensemble (même réseau Docker).

 5. Intégration dans Jenkins

Ton **Jenkinsfile** (aussi à la racine du projet) utilisera ces fichiers :

```
pipeline {
    agent any
    stages {
```

```

stage('Checkout') {
    steps {
        git branch: 'main', url: 'https://github.com/toncompte/tonprojet.git'
    }
}

stage('Build Docker Image') {
    steps {
        sh 'docker build -t myapp:latest .'
    }
}

stage('Run with Docker Compose') {
    steps {
        sh 'docker-compose up -d'
    }
}

stage('Cleanup') {
    steps {
        sh 'docker-compose down'
    }
}
}

```

6. En résumé visuel

Fichier	Rôle	Emplacement	Utilisé par
Dockerfile	Décrit comment construire l'image de ton app	Racine du projet	docker build dans Jenkins
docker-compose.yml	Orchestration de plusieurs conteneurs (app + DB)	Racine du projet	docker-compose up dans Jenkins
Jenkinsfile	Pipeline CI/CD (Build, Test, Analyse, Déploiement)	Racine du projet	Jenkins

Chapitre 4 : La vérification de la qualité du code

❖ Qu'est-ce que SonarQube ?

SonarQube est une plateforme d'analyse de code statique qui mesure la qualité du code : bugs, vulnérabilités, mauvaises pratiques, dette technique, couverture de tests, duplication, etc.

Principales fonctionnalités :

- Analyse de code pour de nombreux langages (Java, JS, Python, C#, ...)
- Quality Gates : règle(s) qui valident/échouent automatiquement une build
- Tableaux de bord et historiques
- Analyse de branches et de Pull Requests (PR)
- Intégration CI (Jenkins, GitHub Actions, GitLab CI...)

Exemple d'un Jenkinsfile avec une stage SonarQube:

```
pipeline {  
    agent any  
    environment {  
        SONARQUBE_TOKEN =  
        credentials('sonar-token-id') // ID
```

```
    }  
    stage('Quality Gate') {  
        steps {  
            waitForQualityGate abortPipeline: true  
        }  
    }  
}
```

du token Jenkins

```

}

stages {
    stage('Build') {
        steps {
            echo 'Building the project...'

            sh './gradlew build' // ou mvn package pour Maven
        }
    }

    stage('SonarQube Analysis') {
        steps {
            withSonarQubeEnv('SonarQube-Server') { // Nom du serveur configuré dans Jenkins

                sh 'sonar-scanner -Dsonar.projectKey=MyProject -Dsonar.sources=src
-Dsonar.host.url=http://192.168.1.45:9000 -Dsonar.login=$SONARQUBE_TOKEN'
            }
        }
    }
}

```

3 Explications

- **withSonarQubeEnv**('SonarQube-Server') : charge les variables d'environnement du serveur SonarQube configuré dans Jenkins.
- **sonar-scanner** : lance l'analyse du code.
 - sonar.projectKey : identifiant unique du projet sur SonarQube.
 - sonar.sources : dossier contenant ton code.
- **waitForQualityGate** : Jenkins attend que SonarQube analyse le projet et renvoie le **Quality Gate** (succès/échec).

2 Exemple d'un Jenkinsfile avec une stage SonarQube

Si tu as déjà un pipeline **declarative**, tu peux ajouter quelque chose comme ceci :

```

pipeline {
    agent any

    environment {
        SONARQUBE_TOKEN = credentials('sonar-token-id') // ID du token Jenkins
    }

    stages {
        stage('Build') {
            steps {
                echo 'Building the project...'
            }
        }
    }
}

```

```

        sh './gradlew build' // ou mvn package pour Maven
    }
}
stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube-Server') { // Nom du serveur configuré dans
Jenkins
            sh 'sonar-scanner -Dsonar.projectKey=MyProject -Dsonar.sources=src
-Dsonar.host.url=http://192.168.1.45:9000 -Dsonar.login=$SONARQUBE_TOKEN'
        }
    }
}

stage('Quality Gate') {
    steps {
        waitForQualityGate abortPipeline: true
    }
}
}
}

```

3 Explications

♦ Explication ligne par ligne :

♦ **withSonarQubeEnv('SonarQube-Server')**

Configure automatiquement l'environnement avec les **paramètres du serveur SonarQube** que tu as ajoutés dans Jenkins (*Manage Jenkins* → *Configure System* → *SonarQube Servers*).

Il fournit les variables comme :

SONAR_HOST_URL

SONAR_AUTH_TOKEN

Ici, 'SonarQube-Server' est le **nom du serveur** défini dans cette configuration Jenkins.

♦ **sh 'sonar-scanner ...'**

C'est la commande qui **lance l'analyse de code SonarQube**.

Option	Rôle
-Dsonar.projectKey=MyProject	Identifie ton projet dans le tableau de bord SonarQube
-Dsonar.sources=src	Indique le dossier contenant le code source à analyser
-Dsonar.host.url=http://192.168.1.45:9000	URL du serveur SonarQube
-Dsonar.login=\$SONARQUBE_TOKEN	Authentification via ton token Jenkins sécurisé

💡 Ainsi, Jenkins analyse ton code (qualité, duplications, bugs, sécurité) et envoie les résultats à SonarQube.

❶ Dans Jenkins

1. Va dans **Manage Jenkins** → **Configure System** → **SonarQube servers**.
2. Clique sur **Add SonarQube** et remplis :
 - **Name** : un nom pour ton serveur (ex : SonarQube-Server)
 - **Server URL** : http://192.168.1.45:9000
 - **Server authentication token** : le token généré dans SonarQube
3. Sauvegarde.

❷ Générer un token dans SonarQube

1. Connecte-toi à http://192.168.1.45:9000
2. Clique sur ton profil → **My Account** → **Security** → **Generate Tokens**
3. Copie ce token pour le mettre dans Jenkins comme **Credential**.

❸ Dans ton Jenkinsfile

Tu utilises le **nom du serveur** que tu as défini (SonarQube-Server) pour te connecter à SonarQube :

```
withSonarQubeEnv('SonarQube-Server') {
    sh 'sonar-scanner -Dsonar.projectKey=MyProject -Dsonar.sources=src
    -Dsonar.login=$SONARQUBE_TOKEN'
}
```

❖ **La qualité du code est cruciale en DevOps pour plusieurs raisons fondamentales**

1. Amélioration de la collaboration et de la communication
2. Réduction des erreurs et des défauts
3. Augmentation de la vitesse de livraison
4. Sécurité renforcée.
5. Meilleure expérience utilisateur

6. Coûts réduits à long terme, 7. Culture de l'excellence

Différences entre SonarQube et SonarCloud

1. Déploiement :

- **SonarQube** : Solution auto-hébergée que vous devez installer et maintenir sur vos propres serveurs.
- **SonarCloud** : Solution hébergée dans le cloud par SonarSource, nécessitant aucune installation ni maintenance de votre part.

2. Coût :

- **SonarQube** : Peut être gratuit ou payant selon les fonctionnalités et le support requis (versions Community, Developer, Enterprise).
- **SonarCloud** : Basé sur un modèle d'abonnement payant, mais propose un plan gratuit pour les projets open source.

3. Mises à Jour :

- **SonarQube** : Les mises à jour doivent être installées manuellement par l'utilisateur.
- **SonarCloud** : Les mises à jour et les nouvelles fonctionnalités sont automatiquement appliquées.

4. Intégration avec VCS :

- **SonarQube** : Intégration avec les systèmes de contrôle de version (VCS) doit être configurée manuellement.
- **SonarCloud** : Intégration facile avec GitHub, Bitbucket, GitLab, et Azure DevOps pour les projets hébergés sur ces plateformes.

5. Scalabilité :

- **SonarQube** : La scalabilité dépend de l'infrastructure et des ressources que vous mettez en place.
- **SonarCloud** : Offre une scalabilité automatique grâce à l'infrastructure cloud de SonarSource.

	SonarQube	SonarCloud
 Installation	Locale, auto-gérée	En ligne, gérée par SonarSource
 Hébergement	Serveur privé	Cloud public
 Contrôle	Total	Limité
 Maintenance	À ta charge	Automatique

	SonarQube	SonarCloud
💰 Coût	Gratuit ou licence	Abonnement à l'usage

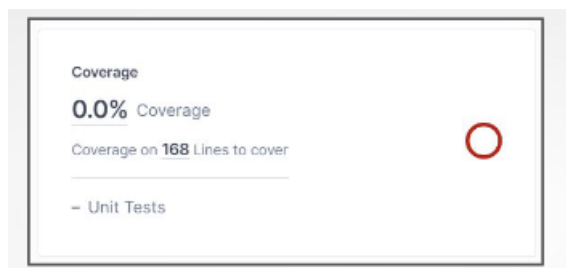


Bug : Un défaut dans le code susceptible de provoquer un comportement indésirable de l'application.

Vulnerability : Une **faiblesse** de sécurité dans notre code.

Hotspots Reviewed (Zones à vérifier) : Parties du code qui nécessitent un examen pour s'assurer qu'elles ne présentent pas de failles de sécurité.

SonarQube indique si le code a été soumis à des outils de test, tels que JUnit.



› **SonarQube lui-même n'effectue pas l'analyse**, mais se repose sur d'autres outils, tels que

JaCoCo. C'est pourquoi notre projet affiche un taux de couverture (Coverage) de 0 %, car JaCoCo **n'a pas été intégré.**

Code Smells (Mauvaises pratiques de codage) : Ce n'est pas une anomalie du code (bug), mais plutôt du code qui peut compliquer la tâche de l'équipe de développement ou de support, par exemple en raison de commentaires excessifs ou des imports inutilisés.

Script installation SonarQube

```
echo "##### Running container sonarqube #####"
docker run -d --name sonarqube -p 9000:9000 -p 9092:9092 sonarqube
```

Kubernetes

Critère	Docker Compose 🐳	Kubernetes (K8s) 🌐
Rôle principal	Orchestrer plusieurs conteneurs sur une seule machine (localement ou sur un petit serveur).	Orchestrer des conteneurs sur plusieurs serveurs (cluster) , de manière distribuée.
Niveau d'utilisation	🏠 Développement local, tests, petits projets.	☁️ Production, grande échelle, haute disponibilité.
Fichier de configuration	docker-compose.yml	Plusieurs fichiers YAML : deployment.yml, service.yml, ingress.yml, etc.
Complexité	Très simple (quelques lignes suffisent).	Complexe : nécessite un cluster, des pods, des services, etc.
Installation	Juste Docker + Docker Compose.	Nécessite un cluster Kubernetes (Minikube, kubeadm, ou cloud comme GKE, AKS, EKS).
Déploiement	Démarre tout avec docker-compose up -d.	Utilise kubectl apply -f <fichiers> pour gérer les ressources.
Mise à l'échelle (scaling)	Manuelle : docker-compose up --scale web=3.	Automatique : Kubernetes gère le scaling et le redémarrage automatique.
Monitoring / Load balancing	Basique ou manuel.	Intégré : auto-scaling, load balancing, redémarrage de pods, etc.
Stockage / Réseaux	Simple, géré par Docker localement.	Géré par Kubernetes via volumes persistants, services, namespaces.
Utilisation typique	Développeur qui teste localement son application multi-conteneurs.	Équipe DevOps qui déploie une application sur le cloud en production.

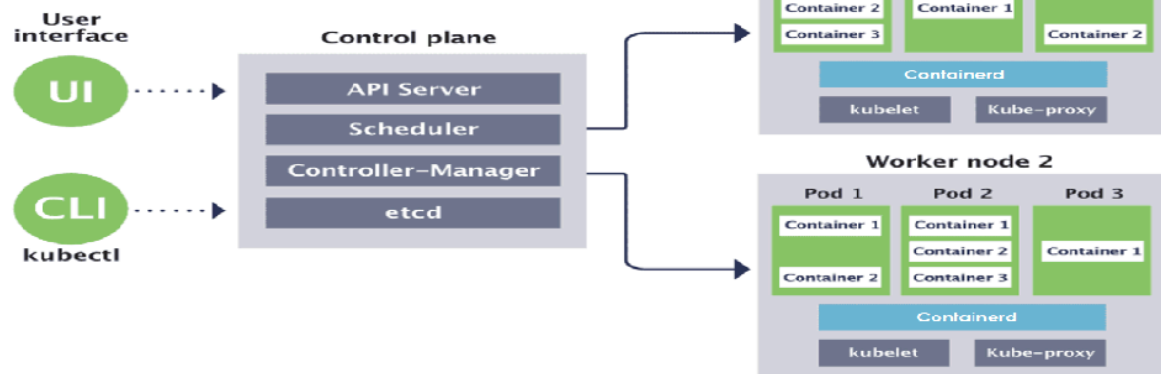
🐳 **Docker Compose** = orchestrateur **simple**, parfait pour le **développement local** et les **petits projets**.

🌐 **Kubernetes** = orchestrateur **avancé**, conçu pour la **production à grande échelle**, la **résilience** et le **scaling automatique**

🌱 **Résumé – Concepts de base de Kubernetes**

Concept	Définition / Rôle principal
Cluster	Ensemble de machines (physiques ou virtuelles) qui exécutent des applications conteneurisées. Il comprend : ♦ 1 nœud maître (Control Plane) → gère et orchestre le cluster. ♦ Plusieurs nœuds de travail (Workers) → exécutent les applications.
Node (Nœud)	Une machine du cluster : ♦ Master Node → contrôle le cluster. ♦ Worker Node → exécute les Pods.
Pod	Plus petite unité déployable dans Kubernetes. Il contient un ou plusieurs conteneurs partageant le même réseau et le même stockage.
Service	Permet d'exposer les Pods via une IP fixe et un nom DNS même si les pods changent. Il peut aussi répartir le trafic (load balancing).
Namespace	Sert à isoler et organiser les ressources du cluster (par exemple : dev, test, prod).
Deployment	Objet qui déploie et gère automatiquement les Pods . Il remplace les Pods défaillants pour assurer la disponibilité.
ReplicaSet	Garantit qu'un nombre fixe de réplicas d'un Pod fonctionnent en permanence. Souvent utilisé à travers un Deployment .
ConfigMap	Contient des données de configuration non sensibles (ex : URL, noms de service).
Secret	Contient des informations sensibles (ex : mot de passe, clé API, token).
Volume / PersistentVolume (PV)	Zone de stockage persistante dans le cluster (les données ne disparaissent pas si le pod redémarre).
PersistentVolumeClaim (PVC)	Requête faite par un Pod pour obtenir un stockage depuis un PV .

Kubernetes architecture



🧠 Composants du nœud maître (Control Plane)

Composant	Rôle principal	Description détaillée
API Server (kube-apiserver)	Point d'entrée de Kubernetes	Reçoit toutes les commandes (via kubectl, dashboard ou API REST). Il valide et enregistre les objets dans etcd. C'est le "cerveau" de la communication.
etcd	Base de données clé-valeur du cluster	Stocke tout l'état du cluster (Pods, Services, ConfigMaps...). C'est la mémoire centrale de Kubernetes.
Controller Manager (kube-controller-manager)	Gère les boucles de contrôle	Surveille l'état réel du cluster et agit pour le faire correspondre à l'état désiré (ex : recréer un Pod supprimé).
Scheduler (kube-scheduler)	Décide sur quel nœud exécuter un Pod	Choisit le meilleur worker node selon les ressources disponibles, les contraintes et les affinités.
Cloud Controller Manager (optionnel)	Intègre Kubernetes au cloud	Gère les ressources spécifiques au fournisseur (ex : création d'instances ou de disques sur AWS, GCP, Azure).

⚙️ Composants du nœud Worker (Node de travail)

Composant	Rôle principal	Description détaillée
Kubelet	Agent principal du nœud	Communique avec le Master via l'API Server, exécute les conteneurs selon les ordres reçus, et surveille leur état.
Kube-proxy	Gère le réseau et le routage	Assure la communication réseau entre les Pods et les Services. Il gère aussi le load balancing interne.

Composant	Rôle principal	Description détaillée
Container Runtime	Exécute les conteneurs	C'est le moteur de conteneurs (ex : Docker, containerd, CRI-O) qui télécharge, démarre et arrête les conteneurs.
Pods	Unité d'exécution réelle	Chaque Pod contient un ou plusieurs conteneurs et tourne sur un Worker node.

Commandes de base Kubernetes

Catégorie	Commande	Description / Usage
 Informations générales	kubectl version	Affiche la version du client et du serveur Kubernetes.
	kubectl cluster-info	Affiche les informations sur le cluster.
	kubectl get nodes	Liste tous les nœuds du cluster.
	kubectl describe node <nom-node>	Détaille la configuration et l'état d'un nœud.
 Gestion des Pods	kubectl get pods	Liste tous les Pods du cluster (dans le namespace actuel).
	kubectl describe pod <nom-pod>	Donne les détails d'un Pod.
	kubectl logs <nom-pod>	Affiche les logs du Pod.
	kubectl exec -it <nom-pod> -- /bin/bash	Ouvre un terminal interactif dans un conteneur.
 Deployments	kubectl get deployments	Liste les Deployments.
	kubectl describe deployment <nom>	Détaille un Deployment.
	kubectl apply -f <fichier.yaml>	Crée ou met à jour un objet à partir d'un fichier YAML.
	kubectl delete -f <fichier.yaml>	Supprime un objet défini dans un fichier YAML.
	kubectl scale deployment <nom> --replicas=3	Change le nombre de Pods dans un Deployment.
 Services & Réseaux	kubectl get services	Liste les Services du cluster.

Catégorie	Commande	Description / Usage
	<code>kubectl describe service <nom></code>	Détaille un Service.
	<code>kubectl expose deployment <nom> --type=NodePort --port=80</code>	Expose un Deployment sous forme de Service.
 Namespaces	<code>kubectl get namespaces</code>	Liste les namespaces.
	<code>kubectl create namespace <nom></code>	Crée un nouveau namespace.
	<code>kubectl delete namespace <nom></code>	Supprime un namespace.
 Configuration et secrets	<code>kubectl get configmaps</code>	Liste les ConfigMaps.
	<code>kubectl get secrets</code>	Liste les Secrets.
	<code>kubectl describe configmap <nom></code>	Détaille un ConfigMap.
 Volumes / Stockage	<code>kubectl get pv</code>	Liste les PersistentVolumes (PV).
	<code>kubectl get pvc</code>	Liste les PersistentVolumeClaims (PVC).
 Autres commandes utiles	<code>kubectl get all</code>	Affiche tous les objets (pods, services, deployments...).
	<code>kubectl delete pod <nom></code>	Supprime un Pod spécifique.
	<code>kubectl rollout restart deployment <nom></code>	Redémarre les Pods d'un déploiement.

🧠 Résumé pratique – workflow typique

#1 Créer un déploiement à partir d'un fichier YAML

```
kubectl apply -f deployment.yaml
```

#2 Vérifier les pods créés

```
kubectl get pods
```

#3 Voir les logs d'un pod

```
kubectl logs <nom-pod>
```

#4 Ouvrir un shell dans un pod

```
kubectl exec -it <nom-pod> -- /bin/bash
```

#5 Supprimer un déploiement

```
kubectl delete -f deployment.yaml
```

 Structure générale d'un fichier YAML (Deployment)

```
apiVersion: apps/v1      # Version de l'API Kubernetes utilisée pour les Deployments
```

```
kind: Deployment         # Type de ressource (ici un "Deployment")
```

```
metadata:                # Informations sur l'objet
```

```
  name: my-deployment    # Nom du déploiement
```

```
  labels:                # Étiquettes pour organiser et filtrer les objets
```

```
    app: my-app
```

```
spec:                    # Spécification du déploiement
```

```
  replicas: 3             # Nombre de Pods à créer
```

```
  selector:              # Sélectionne les Pods gérés par ce déploiement
```

```
    matchLabels:
```

```
      app: my-app
```

```
  template:              # Modèle (template) du Pod à déployer
```

```
    metadata:
```

```
      labels:
```

```
        app: my-app
```

```
    spec:                 # Spécification du Pod
```

```
      containers:        # Liste des conteneurs à lancer dans le Pod
```

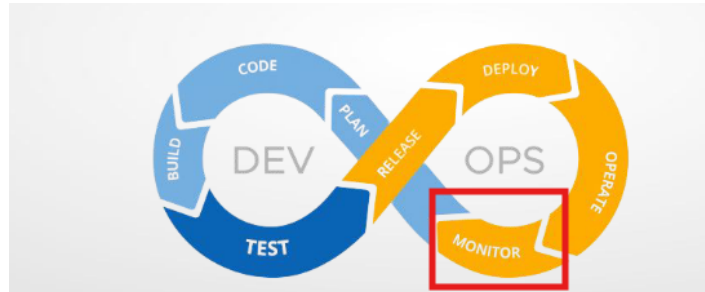
```
        - name: my-container
```

```
          image: nginx:latest    # Image Docker utilisée
```

```
          ports:
```

```
            - containerPort: 80  # Port exposé dans le conteneur
```


Au cœur de la méthodologie DevOps, la surveillance continue est l'outil essentiel qui permet de maintenir un contrôle constant sur les systèmes et les applications, garantissant ainsi une prestation de services ininterrompue.

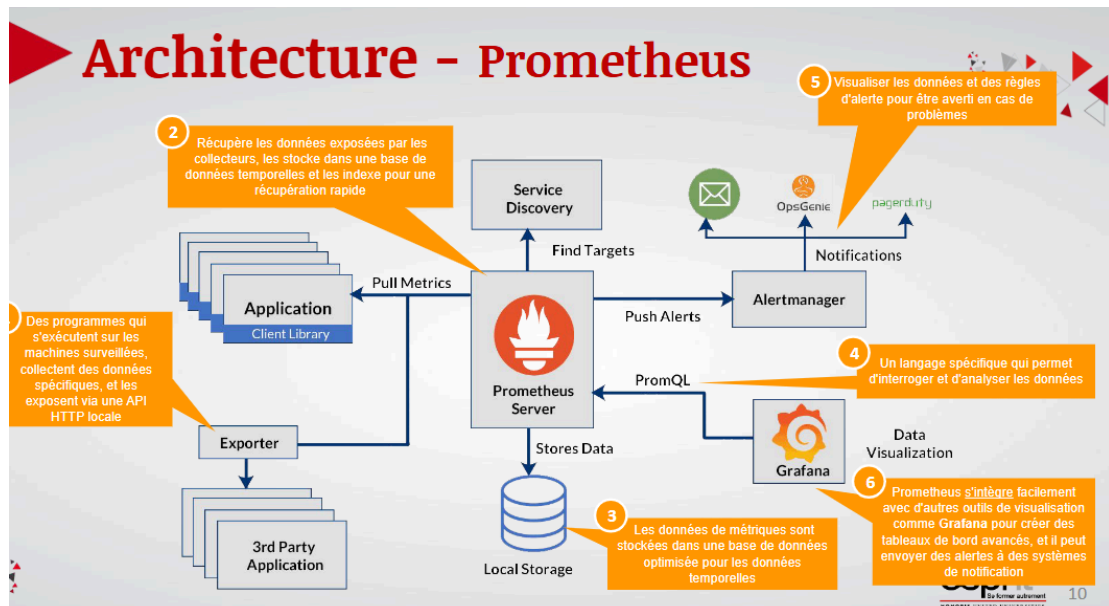


❖ Surveillance Continue – Importance:

Prometheus est un logiciel libre qui surveille les systèmes informatiques, recueille des données en temps réel via HTTP, les stocke dans une base de données performante, et permet d'interroger ces données à l'aide de PromQL, un langage de requête simple

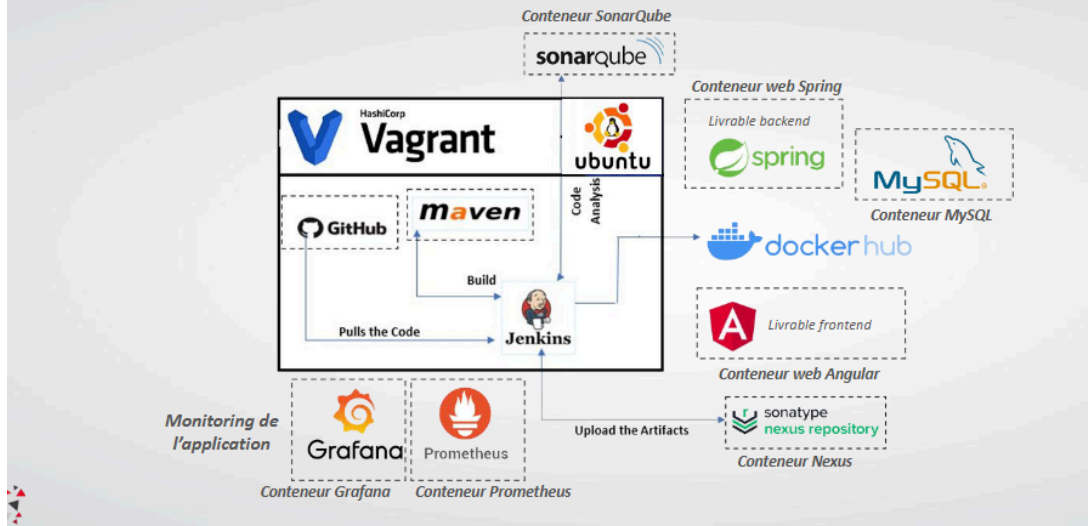


Prometheus n'est pas conçu pour faire de la restitution d'informations sous la forme de tableau de bord. Une bonne pratique est de faire appel à un outil comme Grafana.



- ❖ **Grafana** est un outil open source de monitoring informatique orienté data visualisation. Il est conçu pour générer des dashboards sur la base de métriques et données temporelles.

› Créez des tableaux de bord pour surveiller les divers serveurs ainsi que votre application Spring



1. Exécutez le conteneur grafana:

› Vous pouvez maintenant lancer un conteneur grafana en utilisant la commande :

```
docker run -d --name grafana -p 3000:3000 grafana/grafana
```

```
vagrant@vagrant:~$ docker run -d --name grafana -p 3000:3000 grafana/grafana
Unable to find image 'grafana/grafana:latest' locally
latest: Pulling from grafana/grafana
7264a8db6415: Already exists
3cf7ed17dad5: Pull complete
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
e4a1121b5d45	grafana/grafana	"/run.sh"	4 minutes ago	Up 4 minutes	0.0.0.0:3000->3000/tcp, :::3000->3000/tcp
502b7083a909	prom/prometheus	"/bin/prometheus --c..."	2 hours ago	Up 2 hours	0.0.0.0:9090->9090/tcp, :::9090->9090/tcp

› Vous avez la possibilité d'installer Prometheus et Grafana directement sur votre machine pour effectuer la surveillance.